Section: CSRF, File Upload, and Insecure CAPTCHA

Introduction:

This part focuses on analyzing selected vulnerabilities in DVWA, including CSRF, File Upload, and Insecure CAPTCHA across different security levels (Low, Medium, High).

CSRF (Cross-Site Request Forgery)

Low Level:

At Low security level, the application does not implement any CSRF protection.

Steps:

1. Navigate to the CSRF page.
2. Enter a new password.
3. Submit the request.
4. Capture the request using Burp Suite.

Result:
The password was successfully changed without any validation.

Analysis:
The application does not verify the origin of the request, making it vulnerable to CSRF attacks.

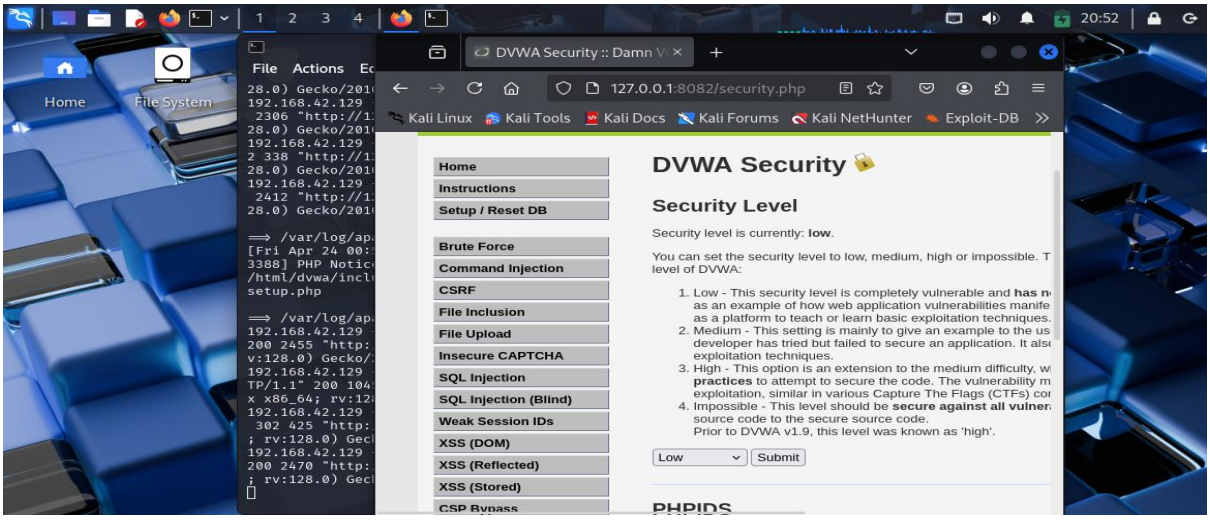

Figure 1: DVWA Security Level set to Low

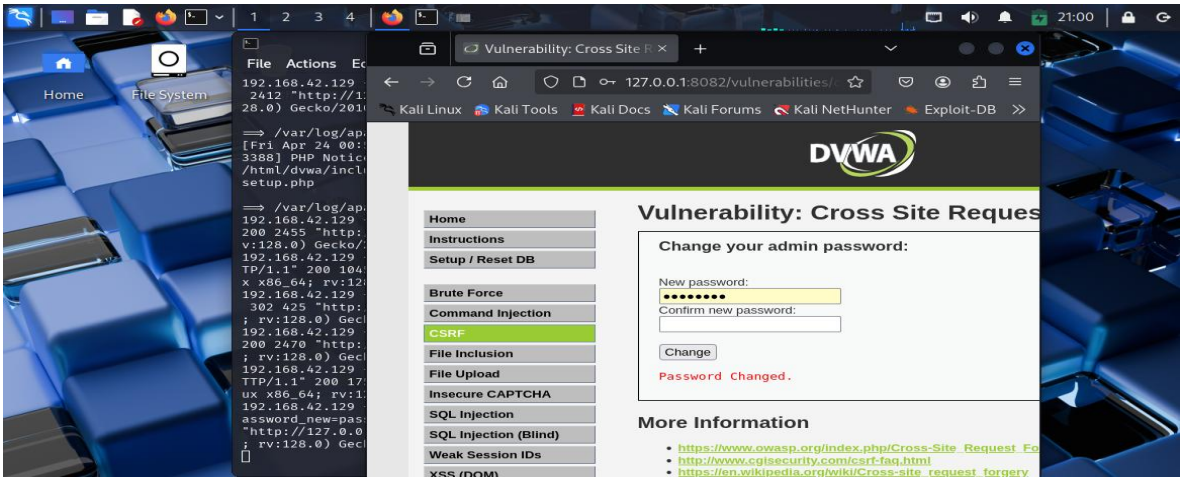

Figure 2:CSRF Low – Password changed successfully

Medium Level:

At Medium security level, the application introduces partial protection using the HTTP Referer header.

Steps:

1. Change security level to Medium.
2. Repeat the password change process.
3. Capture the request using Burp Suite.

Result:
The request includes a Referer header.

Analysis:
Although the application checks the Referer header, this protection is weak and can be bypassed by manipulating request headers.

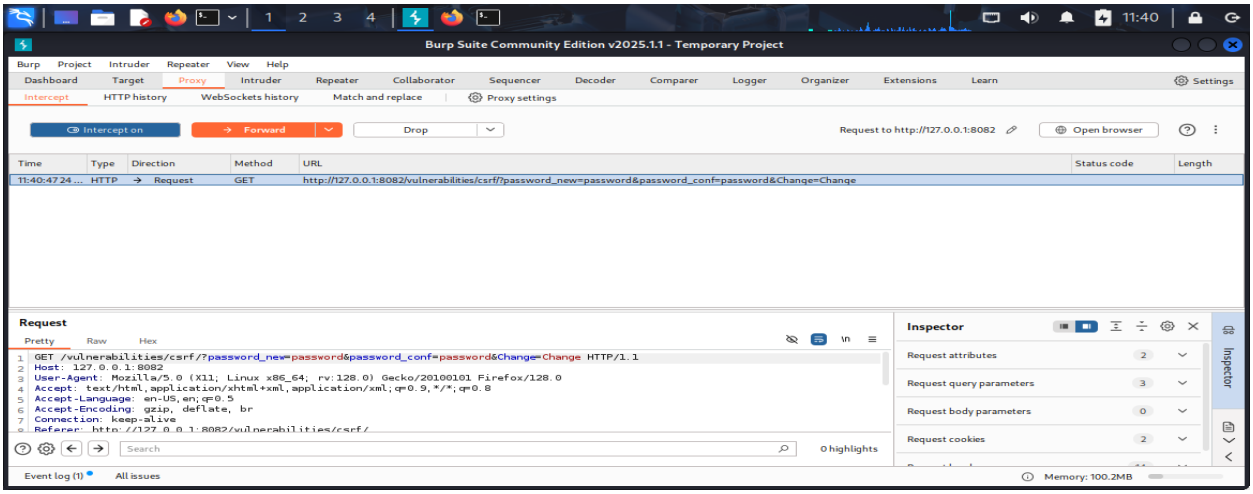

Figure 3:CSRF Medium – Request captured in Burp Suite

High Level:

At High security level, the application uses CSRF tokens for validation.

Steps:

1. Change security level to High.
2. Capture the request using Burp Suite.

Result:
A unique user_token is included in the request.

Analysis:
The use of CSRF tokens significantly improves security by preventing forged requests.

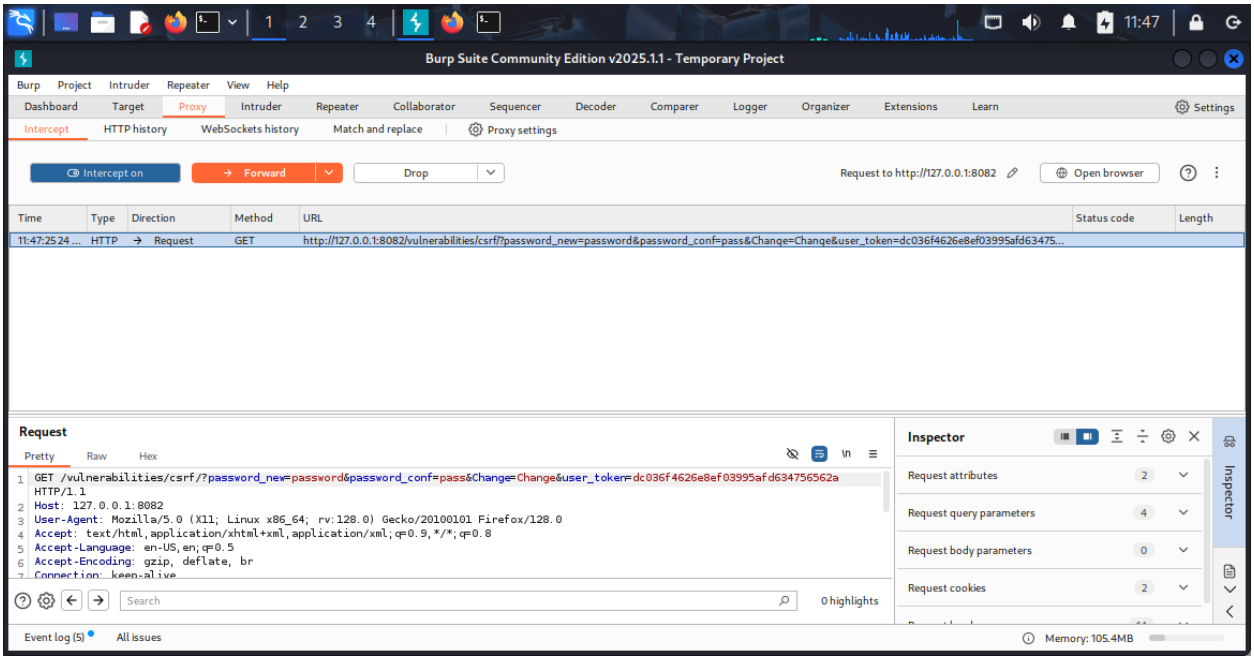

Figure 4:CSRF High – Request containing user_token

File Upload

Low Level:

At Low security level, the application allows unrestricted file uploads.

Steps:

1. Create a malicious PHP file.
2. Upload the file.
3. Access the uploaded file through the browser.

Result:
The file was successfully uploaded and executed, displaying "Hacked".

Analysis:

This vulnerability leads to Remote Code Execution (RCE), which is a critical security risk.

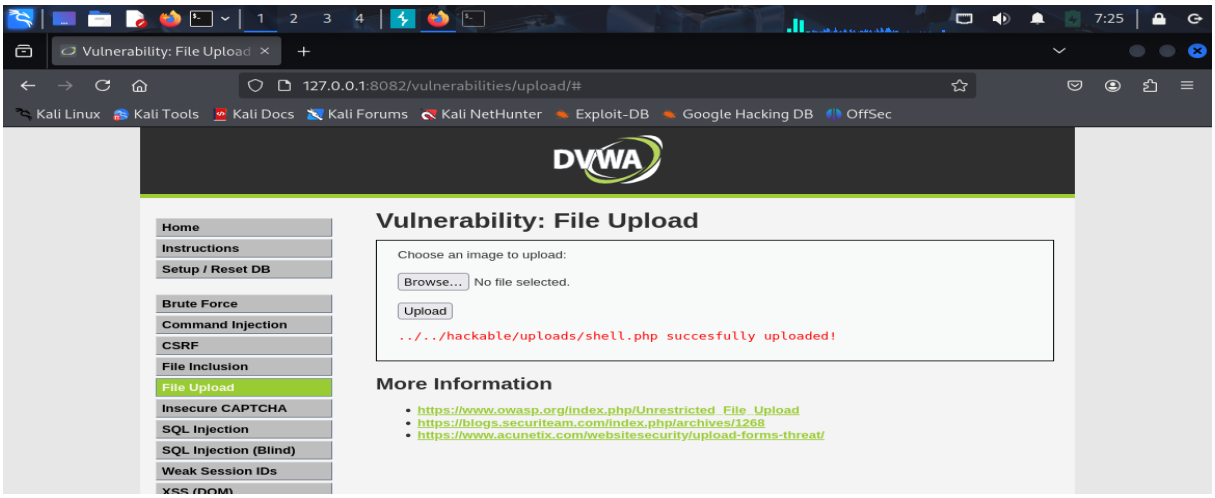

Figure 5:File Upload Low – Malicious PHP file uploaded successfully

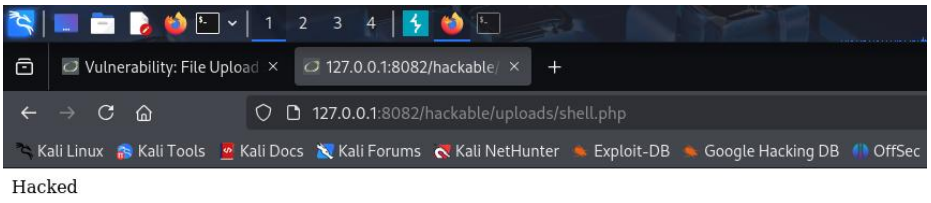

Figure 6: File Upload Low – Uploaded PHP file executed successfully

Medium Level:

At Medium security level, the application applies basic validation based on file extensions.

Steps:

1. Rename the file to shell.php.jpg.
2. Upload the file.

Result:
The file was uploaded but not executed.

Analysis:
The validation is weak because it relies only on file extensions, which can be manipulated.

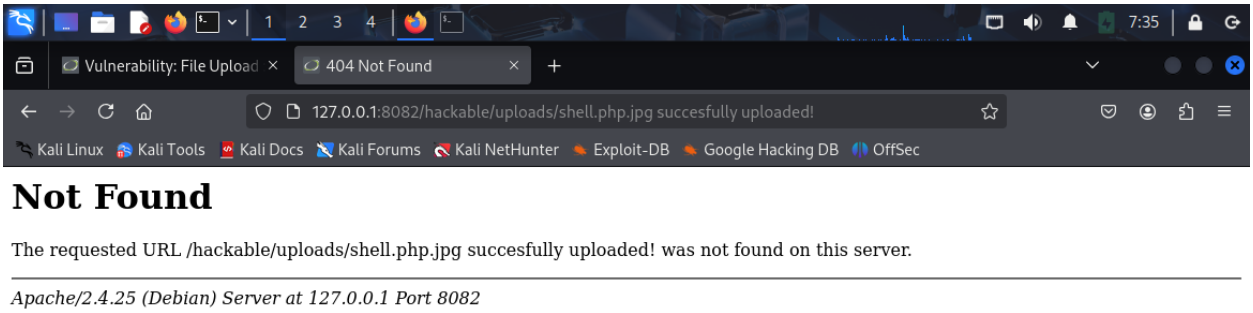

Figure 7:File Upload Medium – Uploaded file not executed

High Level:

At High security level, strong validation is applied to uploaded files.

Steps:

1. Attempt to upload a malicious file.

Result:
The upload was rejected.

Analysis:
Strict validation prevents malicious file uploads, improving application security.

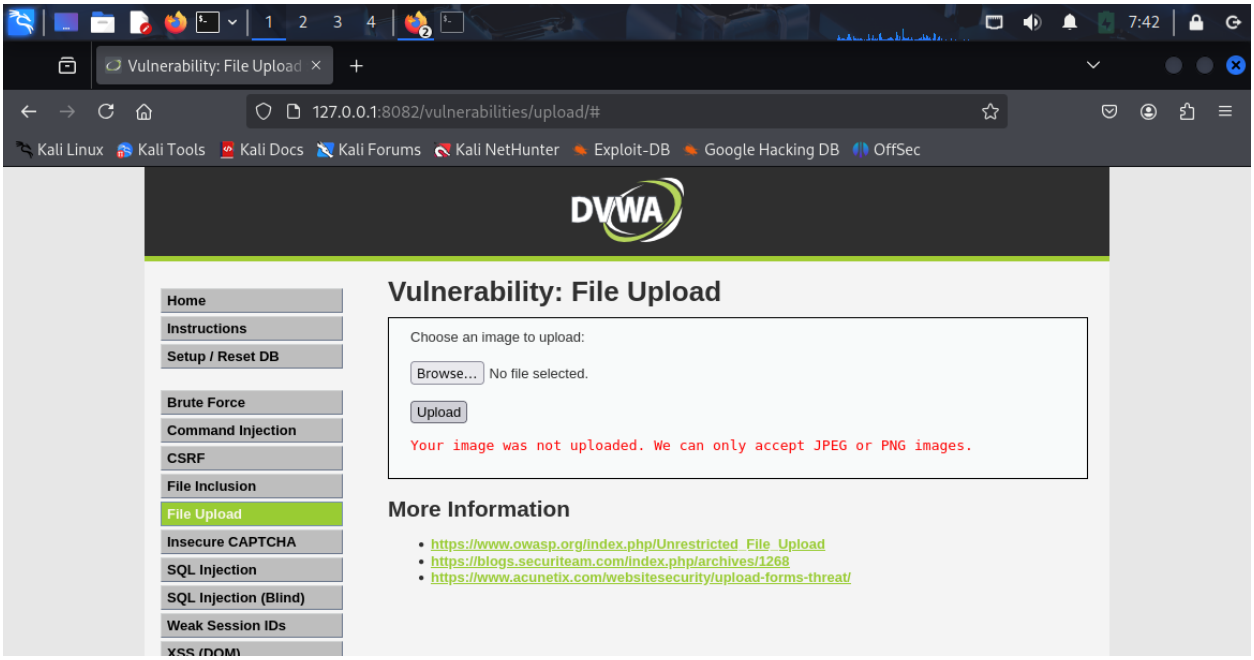

Figure 8:File Upload High – Upload blocked by validation

Insecure CAPTCHA

Description:

The CAPTCHA functionality is not properly configured due to missing API keys.

Analysis:

This indicates a misconfiguration issue where CAPTCHA protection is not enforced correctly. Improper implementation of CAPTCHA can allow attackers to bypass verification mechanisms and perform automated attacks.

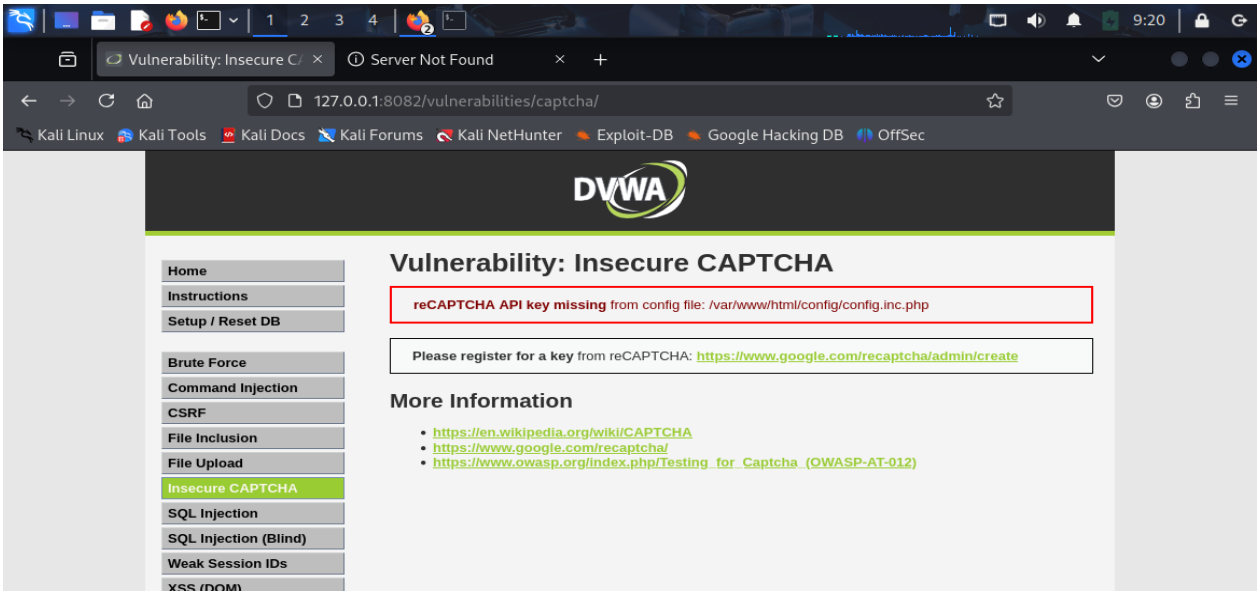

Figure 9: Insecure CAPTCHA – Missing reCAPTCHA API key